01 / Frontend design brief

FireWatch AI • Backend-aligned UI/UX guide • 24 October 2026

Goal: Help a research user find satellite hotspots worth closer inspection, understand their ranking and check the saved evidence. A hotspot is a satellite observation, not a confirmed fire incident.

Current release: Core APIs exist. Baseline scoring is selected; the trained ML candidate is not approved or active. Design from the returned scorer information, not an assumed AI prediction.

1. Keep the experience small

Main navigation: Dashboard, History, Data status. Open hotspot details in a drawer and evidence in a panel or page. Keep the selected region and run visible throughout.

Main journey: Select a region -> view its latest completed results -> filter or explore the map -> open a hotspot -> understand its score -> inspect evidence. History opens the same dashboard in a clearly labelled historical mode.

2. Who supplies what?

Area	User selects	Backend supplies
Context	Existing region, available mode and saved run.	Region bounds, available results, run IDs, cutoff and freshness.
Explore	One priority level, map area, list page and hotspot.	Ranked hotspots, map points, totals, priority and quality flags.
Understand	Open details or expand an explanation.	Measurements, units, missing values, reasons and scorer version.
History	An existing historical run or replay.	Saved scenario name, dates, result context and completed data.
Verify	Open evidence; optionally check referenced files.	Saved evidence, hash comparison, file checks and chain status.

3. Scope and access

• **Viewer:** Browse results and perform read-only verification. No account, upload or manual fire report is required by the current backend.

• **Operator:** Refresh/replay/anchor actions are protected server-side tools. Treat their designs as a separate future back-office flow; end-user login and roles are not implemented.

• Do not add unsupported controls: arbitrary regions/dates, user-entered fire measurements, priority editing, model training, email alerts, evacuation routes or wallet connection.

Always show a short research notice, with the full backend notice available: **"Research prototype. Not for emergency response."**

Backend source: api-v1; /api/v1/regions, /runs, /models/current. Local contract reference: /docs and /openapi.json.

02 / Dashboard & hotspot details

The primary work area. Use backend values; never invent missing results.

Region | Latest results / Historical replay | Saved run

Cutoff + freshness | Total + priority counts | Scoring method

Map + legend

Ranked list -> detail drawer

1. Dashboard requirements

Component	Required content and behaviour
Context bar	Start with the region's latest completed live run, if available. Let users select a saved run or historical replay. Show analysis cutoff and latest observation time separately, with a visible time zone.
Summary + trust	Show total hotspots and Low / Medium / High / Critical counts. Show data completeness, stale/missing-data notices and scoring method. These priorities describe research ranking, not physical fire severity.
Map + ranked list	Show backend point locations, a labelled priority legend and a matching ranked list. A row needs priority, score, latest observation, maximum FRP and quality warning when present. Keep backend ranking order.
Filters + selection	Use one priority selection and an explicit "Apply map area" control, plus Reset filters. Apply the same run, priority and area to map, list and counts. Clicking a marker or row opens the same hotspot drawer.
Updates + limits	List pages default to 50 rows; support pagination (maximum 200). Map responses can be capped at 2,000 points. If truncated, show "Showing X of Y; narrow the area." Checking for new results does not start ingestion. Offer "New results available" without replacing the run being inspected.

Backend source: /hotspots, /hotspots.geojson and /summary require the same run_id; priority and bbox are optional filters. GeoJSON point order is longitude, latitude. All paths use /api/v1.

2. Hotspot drawer

• **Overview:** Priority, score, coordinates, latest observation, number of detections and maximum/summed fire radiative power (FRP, in MW). A short tooltip explains FRP.

• **Why this rank:** Plain-language backend reasons first; expandable technical explanation, scorer/feature versions and analysis cutoff second. Use the selected hotspot's saved scorer, even if the current scorer has changed.

• **Context:** Available weather/climate and activity features, using supplied units. Show missing values as "Not available", never zero. Keep quality flags visible in readable language.

• **Evidence:** Show "View evidence" only when an alert/evidence record exists. Otherwise explain "No saved alert evidence for this hotspot." Provide Close and Copy coordinates.

Score display: A ranking score is not a percentage. Show a probability only when the backend supports it, score_kind is probability and the value is non-null. Current baseline and uncalibrated model outputs must not show a fire-probability gauge. Do not compare scores across different scorer versions.

Backend source: GET /api/v1/hotspots/{snapshot_id}: hotspot, features, units, missing_flags, explanation and optional evidence.

03 / History, status & verification

Supporting views should explain the result without overstating certainty.

1. History

List saved runs by region and mode with cutoff, status, completeness and scorer version. Replay cards also show their saved name and description. Users open completed results; queued/running/failed runs show status rather than an empty result map. There is no arbitrary historical date-range query.

Keep a **Historical replay** label and cutoff visible. Measure freshness against the historical cutoff, not today. Never replace a missing replay with live results.

Backend source: GET /api/v1/runs, /runs/{run_id}, /replays. /replays lists completed saved scenarios, not every scenario that an operator can queue.

2. Data status

Show source status per region/product: Never fetched, Available, Stale, Degraded or Unavailable. Include last successful retrieval and latest observation separately. Show weather retrieval time and scorer availability/limitations. A provider outage must remain distinct from a successful response with no detections.

Backend source: GET /api/v1/status and /models/current. /health/live means the process is alive; /health/ready checks required serving dependencies, not scientific data quality.

3. Evidence panel: three independent checks

Check	Labels to design
Saved evidence local_status	**Unchanged / Changed.** Compare the original saved bytes with the saved hash. Show the saved hash with Copy and an expandable computed hash.
Referenced files artifact_verification	**Not checked / Verified / Mismatch / Partly checked / Unavailable.** Default is Not checked. "Check referenced files" opts into verify_artifacts=true; show per-file results on expansion.
Blockchain anchor_status	**Not anchored / Pending / Match / Mismatch / Unavailable.** Show network kind, chain ID and available transaction/block details. An unavailable lookup is not a mismatch. Label local networks "Local demo".

Use plain helper text: **"These checks show evidence integrity. They do not confirm a fire or prove the model is correct."** A public evidence view may be redacted; do not hash that display to reproduce the original hash. Do not show a single "Verified fire" badge.

Backend source: GET /api/v1/alerts/{alert_id} and /alerts/{alert_id}/verify. Verification reads never enqueue an anchor or sign a transaction.

4. Protected operator concepts - outside viewer MVP

Refresh: Existing enabled region + recent time window (default last 48 hours). **Replay:** Server-allowlisted scenario. **Anchor:** Existing alert only, when chain capability is enabled. Users never provide credentials, digests, model choices or RPC addresses in these forms.

POST /jobs/refresh, POST /jobs/replay and POST /alerts/{alert_id}/anchor return an accepted job, not a finished result. A trusted server handles the operator token and retry identity. Job progress uses Queued / Running / Succeeded / Failed and backend stages, not an invented percentage or ETA.

Backend source: GET /api/v1/jobs/{job_id} is protected. On success, offer the returned result run. Never place ADMIN_API_TOKEN in browser code or browser storage.

04 / States & designer handoff

Design the difficult states as carefully as the populated dashboard.

1. Required states and copy

State	User experience
Loading	Skeletons and a short loading label. Preserve filter selections. Do not flash zero counts or an empty map while loading.
No published run	"No processed results for this region yet." Show region/status context; no fake markers, scores or example results.
Disabled / unavailable mode	Explain that the region is not enabled for new processing, or that this mode has no results. Disable unsupported choices; saved completed results may still be opened.
Valid empty results	"No satellite hotspots found in this run/area." Keep cutoff, source status and filters visible. Do not label the region safe.
Stale / partial data	Keep usable saved results with a clear stale/partial badge, affected source and timestamps. Unavailable values remain unknown.
Run not ready / failed	Explain Processing or Failed. Show available stages and a readable message. Do not show unavailable output as zero, or silently switch runs.
Request error	Keep user context; offer Retry when appropriate. Design invalid-filter feedback (422), missing record (404), service unavailable (503) and access required (401) for private deployments.
Verification problem	Give mismatches a clear warning; show unavailable checks neutrally with Retry where appropriate. Neither state changes the hotspot's scientific priority.

2. Interaction, layout and accessibility

• **Desktop:** Map and list side by side, details in a drawer. **Mobile:** Map/List switch, full-width filters and a bottom sheet or detail page. Preserve the same region, run and selection. Design at 1440 px and 390 px, with a tablet layout check.

• Use readable body text (target 16 px), strong contrast, visible keyboard focus, labelled fields and comfortably sized touch targets (target 44 px). Priorities and statuses need text/icons as well as colour. The list must provide a keyboard alternative to map interaction.

• Support long names, wrapping hashes and large counts. Label time zones and units. Put IDs/hashes in expandable details. A basemap needs provider attribution; backend GeoJSON supplies points, not tiles or fire perimeters.

3. Deliverables for frontend implementation

Provide: Desktop/mobile frames for all views, drawer and evidence; required states; reusable navigation, filters, cards, badges, tables, pagination and feedback; spacing/type/colour tokens; hover/focus/disabled states; and a clickable main journey.

Annotate controls: User input, backend field/source, default, loading/error behaviour and destination. Mark operator controls future/protected. Label sample content; shipped screens must use backend results.

Ready to build when: A user can select one region/run, explore matching map/list/counts, understand a hotspot with missing data, check evidence, open a historical run and recover from errors - without an unsupported API or an invented prediction.